Study and Optimization of SpTRSV Kernels on CPU and GPU

Aditya Padamwar
Dept. of Electrical and Computer Engineering
Northeastern University, Boston, MA
padamwar.a@northeastern.edu

Akash Kotagi
Dept. Electrical and Computer Engineering
Northeastern University, Boston, MA
kotagi.a@northeastern.edu

***Abstract*—Sparse triangular solve (SpTRSV) is a critical operation in scientific computing, arising as the back-substitution phase after sparse LU or Cholesky factorizations. Its optimization is challenging due to strong inter-row data dependencies, low arithmetic intensity, and irregular memory access patterns. This work presents a profiling-guided study of SpTRSV kernel optimization across two CPU platforms (Intel Core i9-14900HX and Xeon Platinum 8276) and three GPU platforms (NVIDIA RTX 4070 Laptop, Tesla P100, and H200).**

On the CPU side, we develop serial optimizations (branch-reduced and unrolled inner loops) and lightweight OpenMP variants, comparing them against Intel oneMKL. Results on 30 real SuiteSparse matrices and six synthetic families show that optimized serial kernels consistently outperform both OpenMP variants and MKL, achieving up to $1.81\times$ speedup over MKL, as threading overhead exceeds the limited available parallelism.

On the GPU side, a profiling-driven development path progresses from a naive baseline through bucketed dispatch that eliminates a $131{,}000$-launch kernel storm, dependency-level scheduling, and family-specialized kernels unified under a learned automatic selector. The learned selector wins on many real matrices on the RTX 4070, P100, and H200 respectively against cuSPARSE, with speedups exceeding $5\times$ on favorable sparsity structures.

All kernels are validated through relative residual norms on the order of 10^{-14}, and every optimization decision is grounded in VTune and Nsight Systems profiling. The results demonstrate that SpTRSV performance is governed by memory behavior, dependency structure, and runtime overhead rather than peak floating-point throughput, and that architecture-specific scheduling is essential for extracting performance from both CPUs and GPUs.

I. Introduction

Sparse triangular solve (SpTRSV) computes the solution vector $\mathbf{x}$ of the system $L\mathbf{x} = \mathbf{b}$, where L is a sparse lower-triangular matrix stored in compressed sparse row (CSR) format. This operation appears as the back-substitution phase in nearly every sparse direct solver and incomplete-factorization preconditioner, making it one of the most frequently invoked kernels in scientific and engineering applications that rely on sparse LU or Cholesky factorizations. Despite its ubiquity, SpTRSV remains one of the hardest sparse kernels to accelerate because of three interacting constraints.

First, the operation is inherently sequential along the dependency chain: row i cannot be completed until every earlier row referenced by its nonzero entries has been solved. This creates a directed acyclic graph of dependencies whose critical-path length is determined by the sparsity pattern of L, and in many practical matrices the resulting parallelism is far narrower than the width of a modern multicore CPU or GPU.

Second, the arithmetic intensity is very low. Each nonzero contributes only a multiply-add and a few index calculations while requiring multiple memory accesses for values, column indices, and the solution vector. Performance is therefore limited by memory bandwidth and latency rather than by peak floating-point throughput.

Third, the access pattern is irregular. Column indices within each row are data-dependent, so the loads from the solution vector exhibit poor spatial locality and are difficult to prefetch. On GPUs this irregularity further undermines coalesced memory access, and on CPUs it defeats hardware prefetchers designed for stride-regular streams.

These characteristics make SpTRSV an instructive case study for understanding the gap between theoretical hardware capabilities and achievable performance on dependency-constrained sparse workloads. Vendor libraries such as Intel oneMKL and NVIDIA cuSPARSE provide tuned SpTRSV implementations, but because they must handle arbitrary sparsity patterns with a single code path, they cannot always exploit structure-specific optimization opportunities that a workload-aware kernel can.

This project investigates SpTRSV optimization through a systematic, profiling-guided methodology on both CPU and GPU platforms. On the CPU side, we develop a series of custom kernels that progress from a basic serial CSR forward solve through branch-reduced and unrolled variants to lightweight OpenMP level-scheduled designs, and compare all of them against Intel oneMKL under a fair single-thread, solve-only timing methodology. On the GPU side, we follow a development path driven entirely by Nsight Systems profiling: starting from a naive single-thread baseline, through cooperative row kernels and bucketed dispatch that eliminate excessive kernel launch overhead, to dependency-level scheduling and a set of family-specialized kernels (RandomAware, PowerLawAware, RegularAware, BandedAware) unified under an automatically learned selector. The evaluation spans two CPU platforms (Intel Core i9-14900HX and Xeon Platinum 8276), three GPU platforms (NVIDIA RTX 4070 Laptop, Tesla P100, and H200), 30 real matrices from the SuiteSparse Matrix Collection, and six synthetic sparsity families at multiple sizes.

The main contributions of this work are as follows. We show that optimized serial CPU kernels consistently outperform both multithreaded OpenMP variants and MKL on SpTRSV, because the synchronization overhead of threading exceeds the narrow parallelism available in most triangular systems. We demonstrate a profiling-driven GPU optimization sequence that reduces kernel launch count from over 131,000 to 841 and progressively shifts the bottleneck from launch overhead to frontier scheduling to family-specific dispatch. We introduce a learned kernel selector for the GPU that matches or beats cuSPARSE on the majority of real matrices tested on the P100 and H200 platforms. Finally, we validate all kernels through relative residual norms on the order of 10^{-14} and ground every optimization decision in quantitative profiling evidence from Intel VTune and NVIDIA Nsight Systems.

II. Background

A. Sparse Triangular Solve Formulation

Sparse triangular solve computes $\mathbf{x}$ from the system $L\mathbf{x} = \mathbf{b}$ [1], where L is an $n \times n$ sparse lower-triangular matrix with a nonzero diagonal. In CSR format, the matrix is represented by three arrays: a row-pointer array `rowPtr` of length $n+1$, a column-index array `colIdx`, and a values array `val`, both of length equal to the number of nonzeros (NNZ). The forward solve processes rows in order from $i = 0$ to $n-1$. For each row i, the algorithm accumulates the dot product of the already-known solution entries referenced by the off-diagonal nonzeros in that row, subtracts this sum from b_i, and divides by the diagonal element:

$$x_i = \frac{b_i - \sum_{j \in \text{colIdx}(i),\ j<i} L_{ij}\, x_j}{L_{ii}}.$$

This formulation makes the dependency structure explicit: row i reads solution entries x_j for $j < i$, so it cannot begin until all such x_j values have been computed. The resulting dependencies form a directed acyclic graph (DAG), whose structure governs the available parallelism [2].

B. Level-Set Parallelism

The dependency DAG of a sparse triangular matrix can be decomposed into levels by a breadth-first traversal [3]. Level 0 contains all rows with no off-diagonal nonzeros below the diagonal (trivially solvable), level 1 contains rows that depend only on level-0 rows, and so on. Within each level, all rows are independent and can be solved in parallel. The width of each level determines the available parallelism, while the total number of levels determines the critical-path length.

In practice, the level structure varies enormously across matrices. Banded and block-structured matrices often have wide levels with hundreds or thousands of independent rows, making them amenable to parallel execution. Chain-like and road-network matrices, by contrast, produce deep, narrow level structures where most levels contain only a few rows, severely limiting parallelism. This variation is a central challenge for SpTRSV optimization: a kernel design that assumes wide levels will waste resources on narrow ones, and a design optimized for sequential execution will underutilize hardware when wide levels are available.

C. CSR Storage Format

The compressed sparse row (CSR) format stores only the nonzero elements of a sparse matrix, making it efficient for row-wise traversal. The `rowPtr` array marks the start and end of each row's nonzeros within the `colIdx` and `val` arrays, so the nonzeros of row i occupy positions `rowPtr[i]` through `rowPtr[i+1]-1`. This layout provides compact storage and enables efficient sequential row access, which aligns well with the forward-solve algorithm. However, the indirect indexing through `colIdx` when accessing the solution vector $\mathbf{x}$ creates irregular memory access patterns that are difficult to optimize on both CPUs and GPUs [4].

D. Performance Characteristics

Sparse triangular solve exhibits very low arithmetic intensity. Each nonzero contributes only a multiply-add and a few index calculations while requiring multiple memory accesses for values, column indices, and the solution vector. As a result, performance is typically limited by memory bandwidth and latency rather than peak floating-point throughput [5]. Furthermore, the irregular access pattern inhibits effective caching and hardware prefetching, especially on GPUs where coalesced memory access is critical for performance.

E. Vendor Baselines

Two vendor-provided SpTRSV implementations serve as baselines in this work. Intel oneMKL provides `mkl_sparse_d_trsv`, which internally performs an analysis phase to determine the level structure and an optimized solve phase [6]. For a fair comparison, we configure MKL in single-thread mode and time only the solve phase, excluding the analysis and setup overhead. On the GPU side, NVIDIA cuSPARSE provides `cusparseSpSV`, which similarly performs an analysis phase followed by a solve phase [7]. cuSPARSE represents the production-grade GPU baseline and is widely used in applications that offload sparse linear algebra to GPUs.

Both libraries must handle arbitrary sparsity patterns through a general-purpose code path, which limits their ability to exploit structure-specific properties of particular matrix families. This generality motivates the development of family-specialized custom kernels that can adapt their scheduling and dispatch strategy to the observed sparsity structure.

III. Experimental Setup

A. Hardware Platforms

All experiments are conducted on five hardware platforms spanning two CPU architectures and three GPU architectures. Table I summarizes the key specifications.

The local machine provides a high-frequency consumer CPU and a mobile GPU for development and initial profiling. The Explorer cluster provides a server-class Xeon CPU and two datacenter GPUs representing different generations of

TABLE I
HARDWARE PLATFORMS USED IN THIS WORK

Platform	Type	Location
Intel Core i9-14900HX	CPU	Local
Intel Xeon Platinum 8276 @ 2.20 GHz	CPU	Explorer
NVIDIA RTX 4070 Laptop (CC 8.9)	GPU	Local
NVIDIA Tesla P100-PCIE-12GB (CC 6.0)	GPU	Explorer
NVIDIA H200 (CC 9.0)	GPU	Explorer

NVIDIA architecture, enabling cross-platform comparisons that reveal how SpTRSV performance scales with hardware capability.

B. Software Stack

All CPU code is written in C++17 and compiled with full optimizations. OpenMP is used for the multithreaded CPU kernels. The CPU vendor baseline uses Intel oneMKL, accessed through `mkl_sparse_d_trsv`. All GPU code is written in CUDA and compiled with the CUDA toolkit. The GPU vendor baseline uses cuSPARSE, accessed through `cusparseSpSV`. Profiling is performed using Intel VTune for CPU analysis and NVIDIA Nsight Systems for GPU analysis [8].

C. Benchmark Matrices

The evaluation uses two categories of input matrices. The first category consists of 30 real sparse matrices obtained from the SuiteSparse Matrix Collection, selected to cover a range of application domains, matrix sizes, and sparsity structures. These matrices span sizes from 138 rows (`bcsstm22`) to 2.39 million rows (`wiki-Talk`) and nonzero counts from 138 (`bcsstm22`) to 5.93 million (`pwtk`). They include structural engineering matrices (the `bcsstk` family), circuit simulation matrices (`scircuit`, `circuit204`, `ASIC_100k`, `ASIC_680k`), social network graphs (`soc-Epinions1`, `soc-sign-epinions`, `wiki-Vote`, `wiki-Talk`), road networks (`roadNet-CA`, `roadNet-TX`), web graphs (`web-Stanford`, `web-BerkStan`), and finite-element meshes (`apache1`, `helm2d03`, `fv1`, `fv2`, `fv3`) [8]. Lower-triangular factors are extracted or constructed from these matrices for the triangular solve.

The second category consists of six synthetic matrix families generated at four sizes each ($n = 1024$, 4096, 16384, 32768), designed to isolate specific sparsity behaviours:

- **Random**: Irregular sparsity with no exploitable structure, producing moderate-width levels with unpredictable column indices.
- **Banded**: Fixed-bandwidth lower-triangular matrices with predictable, localized nonzero patterns that favour cache-friendly traversal.
- **Chain**: Nearly diagonal matrices with minimal off-diagonal fill, producing very deep and narrow level structures with almost no available parallelism.
- **Wide-level**: Matrices designed to produce a small number of very wide dependency levels, maximizing the parallelism available within each level.

TABLE II
SUMMARY OF CUSTOM CPU KERNELS

Kernel	Optimization Idea
Basic	Serial CSR forward solve baseline with minimal overhead and direct dependency-respecting traversal.
BranchReduced	Separates diagonal handling from the hot accumulation loop to reduce branch work in the inner loop.
Unrolled	Adds manual inner-loop unrolling on top of the branch-reduced structure to improve instruction-level efficiency.
ThreshOmp	Lightweight OpenMP level-scheduled kernel that parallelizes only when a dependency level is wide enough to justify the overhead.
ChunkedOmp	Uses coarser-grained OpenMP chunks to reduce scheduling cost compared with per-level parallel execution.
CapChunkO	Chunked OpenMP kernel with a hard cap of 1–2 threads to reduce `libgomp` overhead.

- **Block**: Block-structured lower-triangular matrices with dense diagonal blocks and sparse off-diagonal coupling.
- **Power-law**: Matrices with a skewed degree distribution mimicking social and web graphs, where a few rows have many nonzeros and most rows are sparse.

Together, the real and synthetic sets provide both realistic workload diversity and controlled experiments that isolate the effect of specific sparsity properties on kernel performance.

D. Timing and Validation Methodology

All timing measurements isolate the solve phase from matrix loading, analysis, and setup. Each kernel is executed repeatedly in memory and the median time is reported to reduce noise from system jitter. For the MKL baseline, the analysis phase is performed once and excluded from the timed region, and MKL is configured in single-thread mode to ensure an apples-to-apples comparison with the serial custom kernels. For cuSPARSE, the analysis phase is similarly excluded and only the solve execution is timed.

Numerical correctness is validated by computing the relative residual norm $\|L\mathbf{x} - \mathbf{b}\|_\infty / \|\mathbf{b}\|_\infty$ for every kernel on every matrix. All kernels produce residuals on the order of 10^{-14} to 10^{-16}, confirming that the optimizations do not compromise numerical accuracy. The full residual tables are provided in Appendix A.

IV. CPU KERNEL DESIGN AND OPTIMIZATION

This section describes the custom CPU kernels developed for SpTRSV, organized by the progression that the profiling evidence motivated. Table II summarizes all CPU kernels and their optimization strategies.

A. Serial Kernels

1) Basic: The starting point is a straightforward serial CSR forward solve. For each row i from 0 to $n-1$, the kernel walks the nonzero entries from `rowPtr[i]` to `rowPtr[i+1]-1`, accumulates the product of each off-diagonal value with the corresponding already-solved x_j, subtracts the sum from b_i,

and divides by the diagonal. This kernel has no parallelism, no loop transformations, and no special-case handling, making it the simplest correct implementation and the easiest to validate against the vendor baseline. Despite its simplicity, Basic proved to be remarkably competitive: its minimal control overhead and straightforward memory access pattern allow the hardware prefetcher to operate effectively on the CSR arrays, and the absence of thread management cost gives it an inherent advantage on a workload where the available parallelism is narrow.

2) BranchReduced: The first serial optimization targets the conditional logic inside the inner loop. In the Basic kernel, every nonzero entry must be checked to determine whether it lies on the diagonal or off the diagonal, introducing a branch in the hot path. BranchReduced restructures the loop so that the diagonal element is handled separately after the accumulation loop completes. The inner loop then processes only off-diagonal entries, removing one branch per nonzero from the critical path. This restructuring is a low-risk transformation that preserves the same memory access order and produces identical numerical results. On matrices where the inner loop is tight and the branch predictor is under pressure, BranchReduced provides a measurable improvement. On matrices with very short rows, the overhead of the separate diagonal handling is comparable to the branch cost it removes, and the two kernels perform similarly.

3) Unrolled: The second serial optimization applies manual loop unrolling to the branch-reduced inner loop. Instead of processing one off-diagonal nonzero per iteration, the unrolled kernel processes multiple nonzeros per iteration with explicit index and value loads, reducing loop control overhead and increasing instruction-level parallelism within each iteration. The unrolled kernel retains a scalar cleanup loop for row lengths that are not multiples of the unroll factor. This transformation is most effective on rows with moderate to large numbers of nonzeros, where the reduced loop overhead is amortized over many multiply-add operations. On very short rows the cleanup loop dominates and the benefit diminishes. Across the benchmark set, Unrolled is the strongest serial custom kernel on matrices with denser rows such as road networks and finite-element meshes, while BranchReduced or Basic remain competitive on sparser matrices.

B. OpenMP Kernels

1) Original OpenMP Family: The initial approach to parallelization grouped rows by dependency levels and processed all rows within each level in parallel using OpenMP. Several variants were explored, including basic parallel-for, guided scheduling, and combinations with the branch-reduced and unrolled inner loops. All of these kernels were dramatically slower than the serial baselines. The root cause was structural: the overhead of OpenMP barriers at every level boundary, thread startup and teardown within the runtime, and load imbalance across rows of different lengths far exceeded the computational work saved by parallel execution. On matrices with deep, narrow level structures, the overhead was particularly severe because each level contained too few rows to justify parallel dispatch. This result motivated a complete redesign of the OpenMP strategy.

2) Lightweight OpenMP Redesign: The redesigned OpenMP kernels were built with a single guiding principle: minimize runtime overhead and parallelize only when the expected benefit clearly exceeds the cost. Three kernels emerged from this redesign.

ThreshOmp introduces a threshold on level width. For each dependency level, the kernel checks the number of independent rows. If the count exceeds a tunable threshold, the rows are processed in parallel using OpenMP; otherwise, they are solved serially within the same thread. This avoids the overhead of parallel dispatch on narrow levels, which constitute the majority of levels in most matrices.

ChunkedOmp takes a coarser-grained approach by grouping rows within each level into chunks and assigning chunks to threads rather than individual rows. This reduces the scheduling granularity and lowers the per-row overhead imposed by the OpenMP runtime. The chunk size is chosen to balance load distribution against scheduling cost.

CapChunkO builds on the chunked design but imposes a hard cap of one or two threads regardless of the number of available cores. This was the critical insight from the OpenMP redesign: unrestricted thread counts were the primary cause of the original OpenMP family's poor performance. With a strict thread cap, the `libgomp` runtime overhead is substantially reduced, and the kernel becomes the only OpenMP variant that approaches the serial kernels in performance. Even so, the best serial kernels still win overall because the residual `libgomp` cost, visible in VTune profiles as 17–27% of total CPU time, remains significant relative to the useful row-solve work.

C. MKL Baseline

Intel oneMKL provides a production-grade SpTRSV implementation through `mkl_sparse_d_trsv`. Early experiments compared MKL in its default multithreaded configuration, which produced misleadingly slow results because MKL's internal threading overhead dominated the solve time on this workload. To establish a fair baseline, MKL was reconfigured to single-thread mode and the timing was restricted to the solve phase only, excluding the analysis and inspector-executor setup. Under this configuration, MKL became a technically sound baseline that reflects the quality of its internal serial solve path. On the final benchmark set, MKL won on a subset of matrices but was not the dominant kernel overall, as the custom serial kernels achieved lower overhead on the majority of the tested matrices.

V. GPU Kernel Design and Optimization

This section traces the GPU kernel development path from a correctness baseline to the final learned selector. Each subsection describes what changed, what profiling revealed, and why the next step was taken. Table III summarizes all GPU kernels.

TABLE III
SUMMARY OF CUSTOM GPU KERNELS

Kernel	Optimization Idea
Naive	One thread solves the entire system sequentially. Correctness baseline only.
Coalesced	Block-cooperative row execution with coalesced CSR reads and shared-memory reduction.
ThreadRow	One thread per row for very short rows.
WarpRow	One warp per row with warp-shuffle reduction for medium rows.
BlockRow	One block per row with shared-memory reduction for heavy rows.
AdaptiveRow	Dispatches thread/warp/block kernels per row based on row size.
RowBucketed	Groups rows into thread/warp/block segments and launches one kernel per segment.
LevelAware	Schedules rows by dependency level to expose frontier parallelism.
RandomAware	Specialized for random-irregular frontiers with reduced per-level allocation overhead.
PowerLawAware	Specialized for highly irregular frontiers common in power-law graphs.
RegularAware	Specialized for regular row structures with predictable nonzero patterns.
BandedAware	Tuned for banded matrices where row neighbourhoods are localized.
FamilyAware	Hand-designed dispatcher over the family-specialized kernels.
LearnedAware	Automatic selector using synthetic-derived weights over the family-specialized kernels.

A. Baseline and Cooperative Row Kernels

1) Naive: The GPU development begins with a single-thread kernel in which one CUDA thread solves the entire triangular system sequentially, identical in structure to the CPU Basic kernel. This kernel exists solely to establish correctness: its output is validated against the CPU reference solution and used as the ground truth for all subsequent GPU kernels. It is far too slow to be useful for performance evaluation, but it provides a clean starting point from which every later optimization can be verified.

2) Coalesced: The first performance-oriented GPU kernel uses block-cooperative row processing. Each thread block is assigned a row, and the threads within the block collaboratively traverse the row's nonzero entries, with consecutive threads reading consecutive CSR entries to achieve coalesced global memory access. The partial products are accumulated in shared memory and reduced to a single sum, which is then used to compute the final solution value for that row. Coalesced was the first custom GPU kernel that delivered genuinely competitive performance, and it remained a strong baseline throughout the project. Its strength lies in the combination of coalesced reads from the CSR arrays and efficient shared-memory reduction, which together hide a significant fraction of the memory latency that dominates SpTRSV.

3) ThreadRow, WarpRow, and BlockRow: To understand how row length affects the optimal execution granularity, three specialized row-solve kernels were developed. ThreadRow assigns one thread per row and is suited only to very short rows where the overhead of warp or block coordination would exceed the useful work. WarpRow assigns one warp (32 threads) per row and uses warp-shuffle instructions to perform the reduction without shared memory, making it efficient for medium-length rows. BlockRow assigns one full thread block per row and uses shared-memory reduction, making it appropriate for rows with large numbers of nonzeros. No single row kernel was best for all matrices, confirming that the optimal execution granularity depends on the row-length distribution of the input matrix.

B. Adaptive Dispatch and Launch Overhead

1) AdaptiveRow: The natural next step was to combine the three row kernels into an adaptive dispatcher that selects ThreadRow, WarpRow, or BlockRow for each row based on its length. AdaptiveRow inspects each row's nonzero count and launches the appropriate kernel. However, profiling revealed a severe structural problem: because each row triggered a separate kernel launch from the host, the total number of kernel launches scaled linearly with the number of rows. At $n = 8192$, AdaptiveRow issued 32,780 kernel launches; at $n = 32{,}768$, it issued 131,084 launches. The launch overhead completely dominated the actual compute time, making AdaptiveRow slower than the simpler Coalesced kernel despite making better per-row decisions. This established that the dispatch structure, not the row-kernel choice, was the primary bottleneck.

2) RowBucketed: RowBucketed directly addresses the launch-storm problem identified by profiling. Instead of launching one kernel per row, the kernel groups rows into three buckets based on their nonzero count: short rows assigned to ThreadRow, medium rows to WarpRow, and long rows to BlockRow. Within each bucket, contiguous runs of rows are compressed into segments, and a single kernel launch processes an entire segment. This reduces the launch count by orders of magnitude: at $n = 16{,}384$, RowBucketed issues only 841 launches, and the count remains 841 at $n = 32{,}768$ because the bucket structure is determined by the sparsity pattern rather than the matrix size. This was the largest single structural improvement on the GPU side after Coalesced, and it shifted the performance bottleneck from launch overhead to matrix-wide scheduling and dependency exposure.

C. Level-Aware and Family-Specialized Kernels

1) LevelAware: With launch overhead eliminated, profiling showed that the remaining bottleneck was the order in which rows were processed. RowBucketed processes rows without regard to the dependency DAG, which can require conservative synchronization. LevelAware addresses this by building the dependency-level structure on the host and processing rows level by level. Within each level, the bucketed row-kernel dispatch is used to handle rows of different lengths efficiently. This ensures correctness without atomics while exposing the maximum available parallelism within each level. LevelAware was structurally superior to RowBucketed on matrices with wide dependency frontiers, but it still carried too much per-level overhead for certain matrix families.

2) Family-Specialized Kernels: Profiling and timing analysis of LevelAware across the synthetic families revealed that different sparsity structures required different scheduling strategies. Four family-specialized kernels were developed.

RandomAware targets matrices with irregular sparsity and moderate-width levels by reducing per-level overhead through memory reuse and tuning bucket thresholds.

PowerLawAware targets matrices with highly skewed degree distributions, adjusting resource allocation to balance heavy and lightweight rows.

RegularAware targets matrices with near-uniform row lengths, simplifying dispatch and using a single optimized execution granularity.

BandedAware targets matrices with localized nonzeros near the diagonal, improving memory locality and reuse through block configuration tuning.

3) FamilyAware: FamilyAware provides a hand-designed dispatcher that classifies each input matrix into one of the recognized families based on structural features (average row length, variance, bandwidth, and level width) and routes it to the corresponding specialized kernel.

D. Learned Selector

LearnedAware replaces the hand-designed dispatcher with an automatic selector that chooses among the family-specialized kernels using learned weights. The selector extracts structural features and computes a score for each candidate kernel, selecting the highest-scoring option. The weights were derived from synthetic training data.

A limitation of this design is that the weights were not retrained on real matrices. While the selector generalizes well when real matrices resemble synthetic families, it produces suboptimal choices for mixed or irregular structures. This explains cases where LearnedAware underperforms cuSPARSE, indicating that retraining or using a more expressive model would further improve performance.

VI. Profiling Analysis

Profiling played a central role in guiding every major optimization decision in this project. On the CPU side, Intel VTune was used to identify where time was spent within each kernel and to quantify the overhead introduced by the OpenMP runtime. On the GPU side, NVIDIA Nsight Systems was used to diagnose kernel launch overhead, identify scheduling bottlenecks, and validate that each structural change addressed the intended problem. This section presents the key profiling findings that motivated the kernel development progression described in earlier sections.

A. CPU Profiling with VTune

CPU profiling was performed using VTune's hotspot analysis in a repeated in-memory profiling mode on the `web-Stanford` matrix. This configuration ensures that the solve kernel itself dominates the profile rather than I/O, matrix loading, or process-level scaffolding, providing an accurate view of where time is spent during the actual computation.

1) Serial Kernel Profiles: The three serial kernels exhibit a clean profile in which the solve function accounts for the overwhelming majority of CPU time. Basic spends 1.534 s in `sptrsvCpuBasic`, accounting for 74.5% of total CPU time, with almost no runtime overhead visible elsewhere. BranchReduced spends 1.429 s in `sptrsvCpuBranchReduced` (72.9%), confirming that the branch restructuring provides a modest reduction in solve time while maintaining the same low-overhead profile shape. Unrolled spends 1.710 s in `sptrsvCpuUnrolled` (75.3%), which is slower than both Basic and BranchReduced on this particular matrix. This result illustrates that loop unrolling does not universally improve performance; on `web-Stanford`, whose rows are relatively short and irregularly sized, the unrolled loop's cleanup path and increased code footprint offset the benefit of reduced loop control overhead.

The key takeaway from the serial profiles is that the solve body itself dominates execution time, with negligible overhead from runtime scaffolding. This confirms that the serial kernels are operating close to the hardware limits imposed by the memory access pattern and dependency structure of the matrix, and that further serial improvements would require fundamentally different algorithmic approaches rather than micro-optimizations.

2) OpenMP Kernel Profiles: The lightweight OpenMP kernels show a qualitatively different profile in which the `libgomp` runtime consumes a significant fraction of total CPU time alongside the useful row-solve work.

ThreshOmp spends 2.641 s in `solveUnrolledRow` (51.6% of CPU time), but the `libgomp` helper function consumes an additional 1.404 s (27.4%). The total wall time is therefore substantially higher than the serial kernels, and more than a quarter of the CPU time is spent in OpenMP runtime overhead rather than useful computation.

ChunkedOmp improves on this by reducing the `libgomp` cost to 0.674 s (17.2%), with 2.246 s (57.1%) spent in the row-solve function. The coarser scheduling granularity reduces the number of runtime interactions per level, recovering some of the overhead lost to fine-grained dispatch.

CapChunkO shows a similar profile to ChunkedOmp, with 2.363 s (57.1%) in the row-solve function and 0.710 s (17.2%) in `libgomp`. The thread cap does not eliminate the runtime overhead but prevents it from scaling with the number of available cores, which was the primary failure mode of the original unrestricted OpenMP family.

The progression from ThreshOmp to CapChunkO demonstrates that the lightweight OpenMP redesign successfully moved the time distribution back toward useful row work. However, even in the best case, the `libgomp` overhead remains visible at approximately 17% of total CPU time, which is large enough to keep the OpenMP kernels behind the serial baselines whose profiles show no such overhead.

3) MKL Profile: The MKL solve-only profile shows 1.392 s in `mkl_sparse_d_trsv_i4` (68.6% of CPU time) using a single thread. This confirms that the single-thread, solve-only configuration provides a technically fair comparison, as

the solve call itself dominates the profile rather than analysis or threading infrastructure. MKL's solve time of 1.392 s on `web-Stanford` places it between BranchReduced (1.429 s) and Basic (1.534 s), indicating that MKL's internal serial path is competitive but not dominant on this matrix.

B. GPU Profiling with Nsight Systems

GPU profiling guided the three major structural transitions in the custom kernel development path: from row-by-row dispatch to bucketed execution, from unscheduled to level-aware processing, and from generic to family-specialized kernels.

1) Launch Overhead in AdaptiveRow: The first critical profiling signal was the excessive number of kernel launches produced by the AdaptiveRow dispatcher. At $n = 8{,}192$, Nsight Systems recorded 32,780 kernel launches, and at $n = 32{,}768$, the count rose to 131,084. The Nsight timeline showed that the GPU was idle for a large fraction of the total execution time, waiting for the host to issue the next launch. The compute kernels themselves were completing quickly, but the host-side dispatch loop could not feed them fast enough to keep the GPU occupied. This profiling evidence made it clear that the per-row launch model was fundamentally incompatible with GPU execution, regardless of how well each individual row kernel was tuned.

2) Launch Reduction with RowBucketed: RowBucketed was designed specifically to address the launch-storm problem. After the transition to bucketed dispatch, Nsight Systems confirmed a dramatic reduction: at $n = 16{,}384$, only 841 kernel launches were recorded, and the count remained at 841 at $n = 32{,}768$. The Nsight timeline showed a correspondingly higher GPU utilization, with the compute kernels occupying a much larger fraction of the total execution time and the inter-launch gaps effectively eliminated. This reduction from 131,084 to 841 launches represents a $156\times$ improvement in launch efficiency and was the single largest structural gain on the GPU side.

3) From Launch-Bound to Scheduling-Bound: With launch overhead removed, the profiling focus shifted to how effectively the GPU's parallel resources were utilized across the dependency structure of the matrix. Nsight traces of RowBucketed on matrices with deep level structures showed that many kernel launches processed only a small number of rows, leaving most of the GPU's streaming multiprocessors idle. The bottleneck had shifted from launch overhead to frontier width: the kernel was launching efficiently, but the dependency DAG did not expose enough independent rows at each level to fill the GPU.

This observation motivated the transition to LevelAware scheduling, which explicitly processes rows by dependency level and batches all independent rows within a level into a single dispatch. For matrices with wide frontiers, this change significantly improved GPU utilization. For matrices with narrow frontiers, the improvement was limited by the inherent serial depth of the dependency structure, confirming that some matrices are fundamentally difficult to parallelize regardless of the dispatch strategy.

TABLE IV
CPU REAL-MATRIX WINNER COUNTS (I9-14900HX)

Kernel	Wins (out of 30)
Basic	10
BranchReduced	10
Unrolled	2
ThreshOmp	1
ChunkedOmp	2
CapChunkO	0
MKL	5

4) Profiling-Driven Family Specialization: The final profiling stage compared LevelAware performance across the synthetic families and identified systematic differences in where time was spent. On random matrices, per-level memory allocation and deallocation consumed a disproportionate fraction of execution time, motivating the buffer-reuse optimization in RandomAware. On power-law matrices, the Nsight trace showed that a small number of heavy rows in each level created serialization points where most threads were idle waiting for the longest row to complete, motivating the rebalanced bucket thresholds in PowerLawAware. On banded and regular matrices, the overhead of the general bucketing logic was unnecessary because the row lengths were uniform, motivating the simplified dispatch paths in BandedAware and RegularAware.

This profiling-driven specialization sequence demonstrates a key methodological lesson: each optimization was a response to a measured bottleneck, not a speculative design choice. The progression from launch-bound to scheduling-bound to family-specific tuning reflects the shifting bottleneck that profiling revealed at each stage.

VII. RESULTS

This section presents the experimental results across all CPU and GPU platforms. Condensed summary tables and charts are provided in the main body to highlight key findings; the complete per-matrix data is available in Appendix A (Tables A–I).

A. CPU Real-Matrix Results

Table IV reports the number of matrices on which each kernel achieved the lowest solve time, out of 30 real matrices from the SuiteSparse collection, on the local i9-14900HX.

The serial kernels collectively dominate, winning 22 of 30 matrices. MKL wins on 5 matrices, and the lightweight OpenMP variants account for only 3 wins. Table V highlights representative matrices from different families to illustrate the performance trade-offs.

Several patterns are visible. On large irregular matrices such as `wiki-Talk` and `ASIC_680k`, Basic achieves $1.81\times$ and $1.66\times$ speedup over MKL respectively, because its minimal control overhead is well suited to the long sequential traversal these matrices require. BranchReduced wins on mid-sized structured matrices such as `ASIC_100k` ($1.53\times$ over MKL) and `soc-sign-epinions`, where the reduced branching in the inner loop provides a measurable benefit. Unrolled is

TABLE V
CPU REAL-MATRIX SOLVE TIMES (MS) ON I9-14900HX — SELECTED MATRICES

Family	Matrix	N	Basic	BranchRed	Unrolled	CapChunkO	MKL	Best	×MKL
random_irreg.	ASIC_100k	99 K	1.083	**1.030**	1.044	1.356	1.575		1.53
wide_level	web-Stanford	282 K	**3.139**	3.544	3.990	3.949	3.638		1.16
power_law	soc-sign-epin.	132 K	1.316	**1.199**	1.273	1.545	1.193		1.00
extra	pwtk	218 K	**3.797**	4.553	4.106	4.664	4.411		1.16
extra	helm2d03	392 K	1.732	1.575	**1.558**	2.828	1.676		1.08
extra	ASIC_680k	683 K	**2.598**	2.751	2.791	6.566	4.312		1.66
very_large	roadNet-TX	1.39 M	9.195	9.413	**9.131**	27.05	12.43		1.36
very_large	wiki-Talk	2.39 M	**9.771**	9.959	16.16	24.16	17.67		1.81

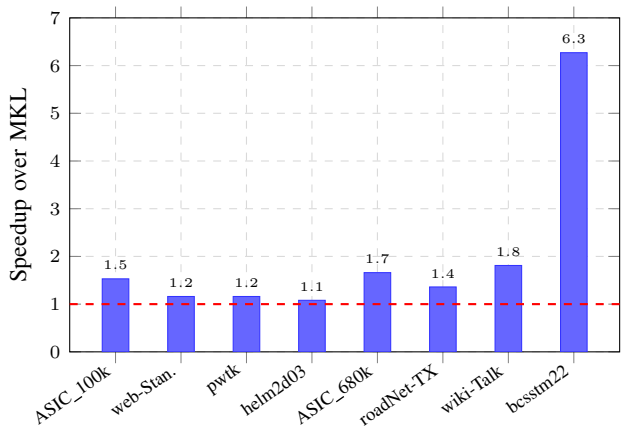

Fig. 1. Speedup of best custom CPU kernel over MKL on selected real matrices (i9-14900HX). Values above the dashed line at 1× indicate the custom kernel is faster than MKL.

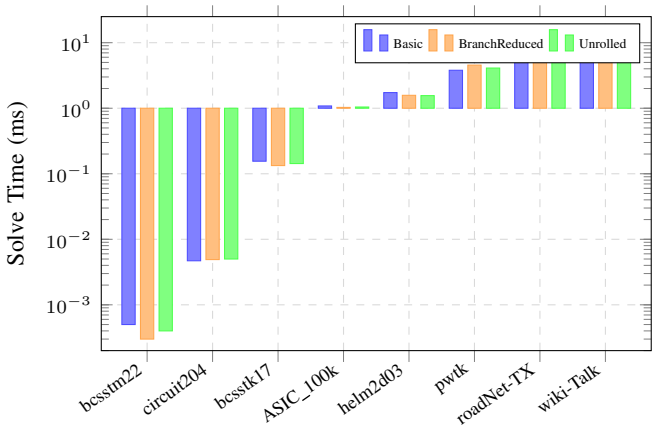

Fig. 2. Comparison of serial CPU kernels on selected real matrices (i9-14900HX, log scale). No single serial kernel dominates across all matrix structures.

strongest on matrices with denser rows such as `helm2d03` and `roadNet-TX`. The OpenMP kernels are almost always slower, and MKL is competitive but rarely the fastest. The full 30-matrix performance table is provided in Appendix A, Table A.

Fig. 1 shows the speedup of the best custom kernel over MKL for the selected matrices.

Fig. 2 compares the three serial kernels on eight representative matrices to show when each variant wins.

B. CPU Synthetic Results

The synthetic benchmark sweep reveals where each kernel family is strongest. Fig. 3 shows the wide-level family at $n = 32{,}768$, which is the only synthetic family where the OpenMP kernels decisively outperform both the serial kernels and MKL.

On the wide-level family at $n = 32{,}768$, CapChunkO solves in 0.288 ms compared to MKL's 0.668 ms (2.32× speedup), because the wide dependency levels provide enough independent rows to justify the threading overhead. On all other families—random, banded, chain, block, and power-law—the serial kernels or MKL remain the fastest, confirming that SpTRSV parallelism is profitable on the CPU only when the level structure is sufficiently wide.

Fig. 4 shows how the best custom kernel and MKL scale across matrix sizes for the banded and random families.

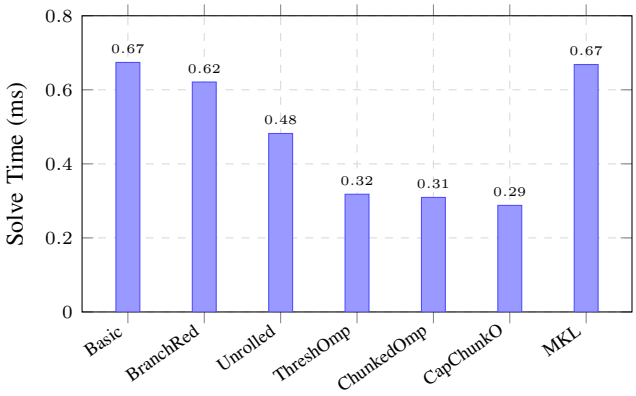

Fig. 3. CPU synthetic wide-level family at $n = 32{,}768$ (i9-14900HX). CapChunkO achieves 2.32× speedup over MKL, the strongest OpenMP result in the entire benchmark.

The complete synthetic performance and residual tables are provided in Appendix A.

C. GPU Real-Matrix Results

Table VI summarizes the LearnedAware vs. cuSPARSE comparison across all three GPU platforms on the 30 real matrices.

The learned selector's effectiveness increases with GPU capability: it wins 30% of matrices on the RTX 4070 but

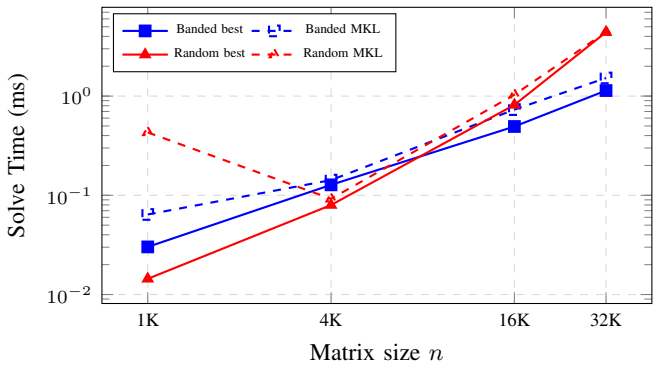

Fig. 4. CPU synthetic scaling: best custom kernel vs MKL for banded and random families across sizes $n = 1024$ to 32,768 (i9-14900HX).

TABLE VI
GPU LEARNEDAWARE VS CUSPARSE: WINNER COUNTS ON REAL MATRICES

Platform	Learned Wins	cuSPARSE Wins	Total
RTX 4070 Laptop	9	21	30
Tesla P100	15	15	30
NVIDIA H200	17	13	30

57% on the H200. This trend is consistent with the hypothesis that higher-end GPUs better tolerate the selector's dispatch overhead and more effectively expose the parallelism in the family-specialized kernels.

Table VII shows representative matrices where LearnedAware strongly outperforms cuSPARSE and where it loses, on the H200.

When LearnedAware wins, the margins are substantial: on `ASIC_100k` it is 5× faster than cuSPARSE, and on `ASIC_680k` it achieves a 4.1× advantage. The consistent failure cases are road networks (`roadNet-CA`, `roadNet-TX`), regular grids (`apache1`, `fv1/fv2/fv3`), and some circuit matrices, where the learned selector misclassifies the sparsity family because the weights were trained on synthetic data only.

Fig. 5 compares LearnedAware and cuSPARSE on selected real matrices on the RTX 4070.

Fig. 6 shows the same comparison on the H200, the strongest platform for LearnedAware.

Fig. 7 shows the LearnedAware/cuSPARSE ratio for all 30 real matrices on the H200. Values below 1.0 indicate that LearnedAware is faster.

D. GPU Synthetic Results

Fig. 8 shows the LearnedAware vs. cuSPARSE speedup by synthetic family at $n = 32{,}768$ on the RTX 4070. Values above 1.0 indicate that LearnedAware is faster.

LearnedAware achieves its strongest advantages on wide-level (8.58×) and chain (6.86×) families, where the level-aware scheduling provides a large structural benefit over cuSPARSE's general-purpose dispatch. It is also strong on banded matrices (3.29×). cuSPARSE wins on random (0.73×) and power-law (0.54×) at $n = 32{,}768$, where the synthetic-

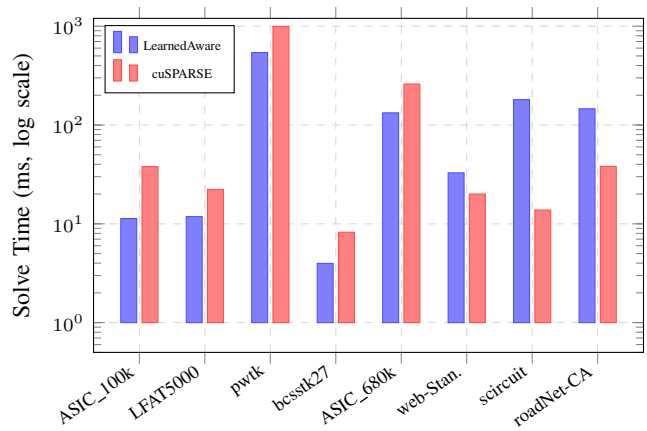

Fig. 5. LearnedAware vs cuSPARSE on selected real matrices — RTX 4070 Laptop (log scale).

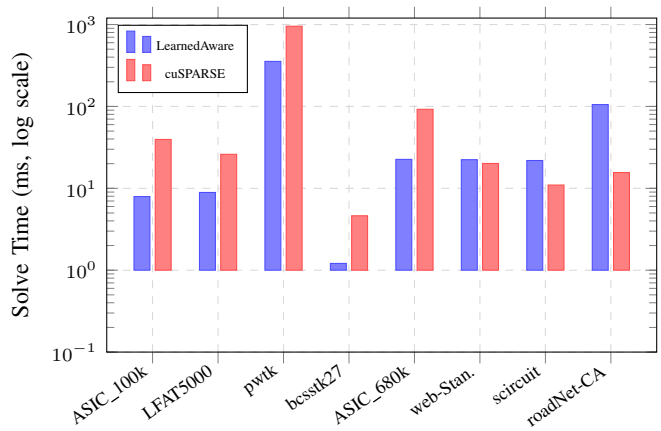

Fig. 6. LearnedAware vs cuSPARSE on selected real matrices — NVIDIA H200 (log scale).

derived selector weights do not produce the optimal kernel choice for these larger problem sizes.

Fig. 9 shows the full kernel progression on the H200 for the random synthetic family, illustrating how each optimization stage reduced solve time.

The complete GPU synthetic tables are provided in Appendix A, Tables D, G, and I.

E. Cross-Platform GPU Comparison

Fig. 10 compares the LearnedAware/cuSPARSE ratio for six representative matrices across all three GPU platforms.

Two patterns are consistent across platforms. First, LearnedAware strongly outperforms cuSPARSE on circuit and irregular matrices (`ASIC_100k`, `ASIC_680k`, `pwtk`) regardless of the GPU, because the family-specialized kernels exploit structural properties that cuSPARSE's general path cannot. Second, cuSPARSE consistently wins on road networks (`roadNet-CA`) and regular grids (`apache1`), where the learned selector misclassifies the matrix family due to its synthetic-only training data. On `web-BerkStan`, the result flips between platforms: cuSPARSE wins on the RTX 4070

TABLE VII
GPU REAL-MATRIX SOLVE TIMES (MS) ON NVIDIA H200 — SELECTED MATRICES

Family	Matrix	N	LearnedAware	cuSPARSE	Ratio	Winner
random_irreg.	ASIC_100k	99 K	**7.93**	39.55	0.20	Learned
banded	LFAT5000	20 K	**8.90**	26.03	0.34	Learned
extra	pwtk	218 K	**354.88**	954.68	0.37	Learned
extra	ASIC_680k	683 K	**22.56**	92.30	0.24	Learned
extra	bcsstk27	1.2 K	**1.21**	4.62	0.26	Learned
extra	apache1	81 K	42.69	**3.61**	11.84	cuSPARSE
extra	roadNet-CA	1.97 M	105.38	**15.58**	6.77	cuSPARSE
very_large	roadNet-TX	1.39 M	74.45	**10.83**	6.87	cuSPARSE
extra	fv1	9.6 K	7.06	**2.86**	2.47	cuSPARSE

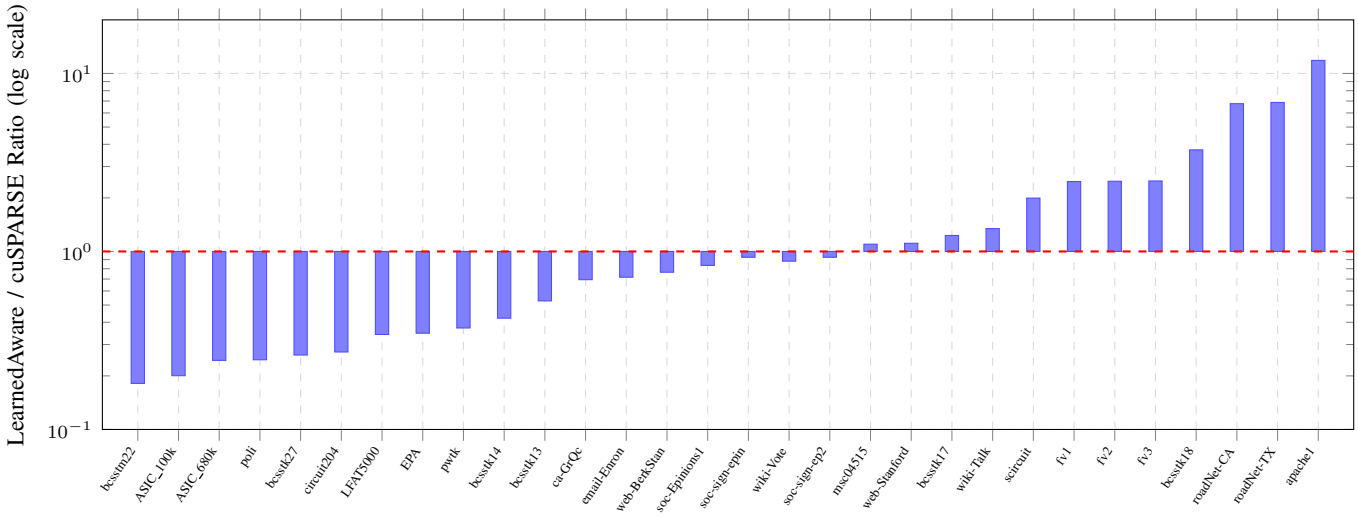

Fig. 7. LearnedAware / cuSPARSE timing ratio for all 30 real matrices on the H200. Bars below the dashed line at ratio = 1.0 indicate LearnedAware is faster. The matrices are sorted by ratio from best (left) to worst (right) for LearnedAware.

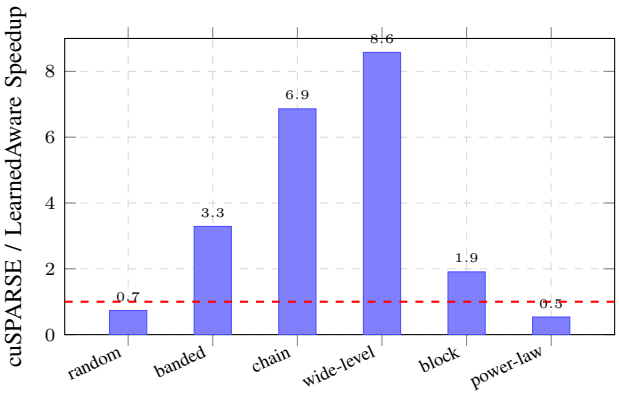

Fig. 8. LearnedAware speedup over cuSPARSE by synthetic family at $n = 32{,}768$ on RTX 4070. Values above the dashed line indicate LearnedAware is faster.

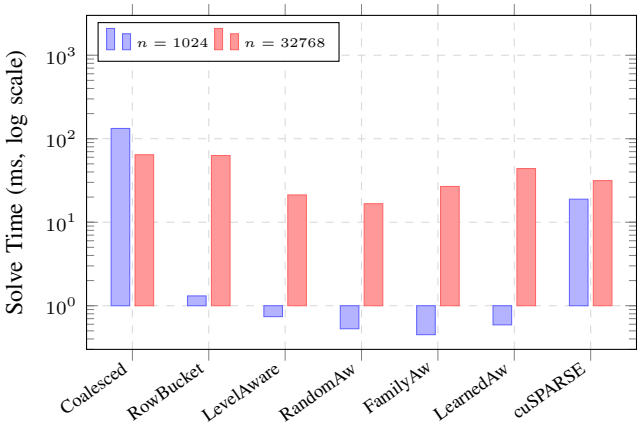

Fig. 9. GPU kernel progression on H200 — random synthetic family. Each bar shows the solve time at a given optimization stage. The improvement from Coalesced to RowBucketed to RandomAware reflects the profiling-guided restructuring described in Section 6.

(ratio 1.46) but LearnedAware wins on the P100 (ratio 0.74) and H200 (ratio 0.76), suggesting that the higher-end GPUs better tolerate the selector's overhead on this borderline case.

F. CPU vs GPU Comparison

Fig. 11 compares the best CPU kernel time (i9-14900HX) against the best GPU option (minimum of LearnedAware and cuSPARSE on H200) for matrices that appear in both

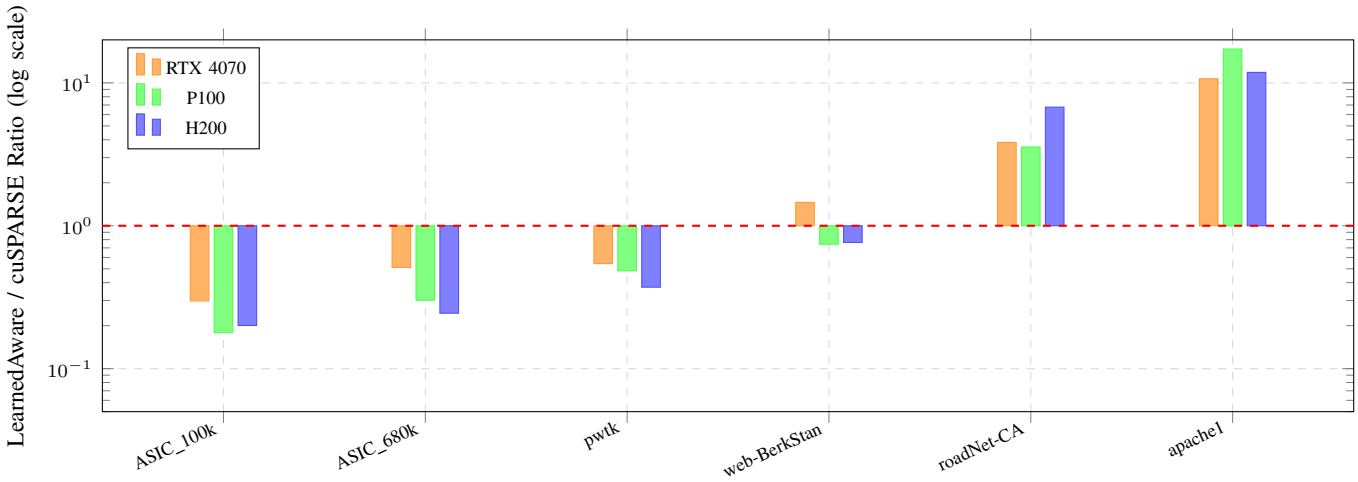

Fig. 10. Cross-platform comparison of LearnedAware / cuSPARSE ratio for selected real matrices. Bars below the dashed line at ratio = 1.0 indicate LearnedAware is faster. LearnedAware's wins are consistent across platforms on circuit and irregular matrices; its losses on road networks and regular grids are also consistent.

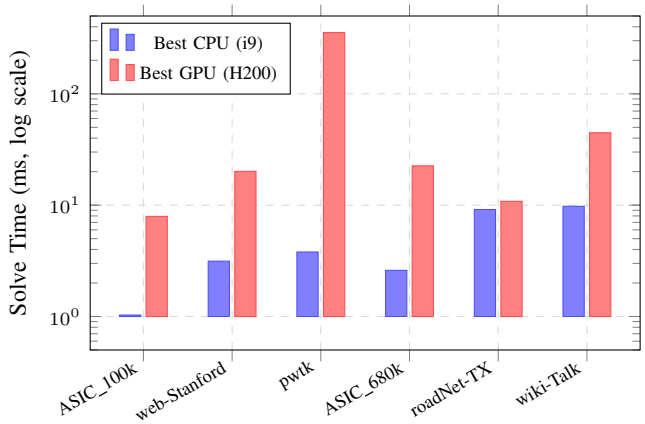

Fig. 11. Best CPU (i9-14900HX) vs best GPU (H200) solve times on shared matrices. The CPU is faster on most matrices due to SpTRSV's limited parallelism and the GPU's kernel launch and synchronization overhead.

benchmark sets.

A striking result is that the optimized CPU serial kernels are faster than the GPU on most of these matrices. On `ASIC_100k`, the CPU solves in 1.03 ms while the best GPU option requires 7.93 ms. On `wiki-Talk`, the CPU (9.77 ms) is 4.6× faster than the GPU (44.71 ms). This is a direct consequence of SpTRSV's limited parallelism and low arithmetic intensity: the GPU's massive thread count cannot compensate for the kernel launch overhead, synchronization cost, and memory latency penalty on this dependency-constrained workload. The GPU approaches CPU performance only on matrices like `roadNet-TX`, where the CPU (9.13 ms) and GPU (10.83 ms) are within 20% of each other.

G. Numerical Validation

Table VIII presents representative residual norms confirming numerical correctness across all kernel families.

TABLE VIII
REPRESENTATIVE RELATIVE RESIDUAL NORMS ($\|Lx - b\|_\infty / \|b\|_\infty$)

Matrix	Basic	Unrolled	CapChunkO	MKL
ASIC_100k	1.92e-14	1.39e-14	1.39e-14	1.52e-14
web-Stanford	2.76e-14	1.94e-14	1.94e-14	2.05e-14
pwtk	6.66e-14	4.04e-14	4.04e-14	2.50e-14
wiki-Talk	4.66e-14	4.39e-14	4.39e-14	8.29e-14
roadNet-CA	4.73e-14	4.23e-14	4.23e-14	9.74e-14

All residuals are on the order of 10^{-14} to 10^{-16}, confirming that the optimizations do not compromise numerical accuracy. The Unrolled and OpenMP kernels produce identical residuals because they share the same unrolled inner-loop implementation. MKL occasionally produces slightly larger residuals (e.g., 9.74×10^{-14} on `roadNet-CA`) due to differences in accumulation order, but all values remain well within double-precision tolerance. Complete residual tables for all matrices are provided in Appendix A.

VIII. CONCLUSION

This project investigated the optimization of sparse triangular solve across two CPU and three GPU platforms through a systematic, profiling-guided methodology. The combined results lead to four central conclusions.

First, **serial optimization is the most effective CPU strategy for SpTRSV**. The custom serial kernels—Basic, BranchReduced, and Unrolled—collectively won 22 of 30 real matrices on the i9-14900HX, with the best custom kernel achieving up to 1.81× speedup over Intel oneMKL. The OpenMP variants, even after a complete lightweight redesign that reduced `libgomp` overhead from 27% to 17% of CPU time, could not overcome the narrow parallelism inherent in most triangular systems. The only exception was the wide-level synthetic family, where CapChunkO achieved 2.32× speedup over MKL, but this represents a best-case scenario that rarely occurs in practice.

Second, **profiling-driven restructuring is more valuable than micro-optimization on the GPU**. The single largest GPU improvement came not from tuning row-kernel thresholds or register usage but from eliminating the kernel launch storm exposed by Nsight Systems: replacing row-by-row dispatch (131,084 launches at $n = 32{,}768$) with bucketed execution (841 launches) produced a $156\times$ improvement in launch efficiency and fundamentally changed the performance bottleneck from launch overhead to frontier scheduling.

Third, **matrix-structure-aware dispatch provides real benefits but requires accurate classification**. The family-specialized GPU kernels (RandomAware, PowerLawAware, RegularAware, BandedAware) outperformed the generic LevelAware kernel on their respective matrix families, and the learned selector that combined them won on the majority of real matrices on the P100 (15/30) and H200 (17/30). However, the selector's synthetic-derived weights caused systematic misclassification on road networks and regular grids, where cuSPARSE remained significantly faster.

Fourth, **SpTRSV performance is governed by memory behaviour, dependency structure, and runtime overhead rather than peak floating-point throughput**. The optimized CPU serial kernels were faster than the best GPU option on most shared matrices, because SpTRSV's low arithmetic intensity and strong row dependencies prevent the GPU from exploiting its massive parallelism. The GPU approached CPU performance only on matrices where the dependency structure exposed sufficiently wide frontiers.

IX. Future Work

Several directions could extend the results of this project.

Retraining the learned selector on real matrices. The current GPU selector uses weights derived exclusively from synthetic matrix families. Retraining on the 30 real SuiteSparse matrices, or on a larger real-matrix training set, would likely improve classification accuracy on the road-network and regular-grid families where the selector currently fails. A more expressive model, such as a small decision tree or a two-layer neural network, could capture nonlinear interactions between structural features that the current linear model misses.

Exploring additional sparse matrix formats. All experiments in this work use CSR storage. Compressed Sparse Column (CSC) format is more natural for backward substitution with upper-triangular matrices, and hybrid formats such as ELL or blocked CSR could improve memory coalescing on the GPU for matrices with uniform row lengths. Evaluating the kernels under alternative formats would clarify how much of the observed performance is format-dependent.

Cycle-accurate simulation and energy analysis. The current evaluation uses wall-clock timing on real hardware, which captures end-to-end performance but does not isolate the contributions of cache behaviour, memory bandwidth, and instruction-level parallelism. Running the kernels in a cycle-accurate simulator such as gem5 or GPGPUSim would provide more detailed insight into the microarchitectural bottlenecks. Energy measurements would also be valuable for understanding the power-performance trade-off between CPU serial execution and GPU parallel execution.

Broader benchmark coverage. The current evaluation uses 30 real matrices and 6 synthetic families. Extending the benchmark set to include matrices from SPEC CPU, PARSEC, and large-scale graph-processing workloads would test the generality of the kernel designs and the learned selector across a wider range of sparsity patterns and application domains.

Integration with iterative solvers. SpTRSV is most commonly used as a component of iterative solvers with incomplete factorization preconditioners, where it is called repeatedly with the same matrix but different right-hand-side vectors. Evaluating the amortized performance of the custom kernels within a full solver loop, including the cost of the analysis phase, would provide a more complete picture of the practical benefit.

Task Division

Project Title: Study and Optimization of SpTRSV Kernels on CPU and GPU

Aditya Padamwar — Contributions

- Designed and implemented GPU kernels:
  - Naive, Coalesced, ThreadRow, WarpRow, BlockRow
  - AdaptiveRow, RowBucketed, LevelAware
  - Family-specialized kernels and LearnedAware selector
- Developed CPU serial (non-OpenMP) kernels:
  - Basic, BranchReduced, Unrolled
- Built and evaluated synthetic benchmark suite:
  - Random, Banded, Chain, Wide-level, Block, Power-law families
- Performed GPU profiling and analysis:
  - Used NVIDIA Nsight Systems
  - Identified kernel launch bottlenecks and scheduling inefficiencies
- Contributed to performance analysis and result interpretation (GPU and synthetic benchmarks)

Akash Kotagi — Contributions

- Designed and implemented CPU OpenMP kernels:
  - ThreshOmp, ChunkedOmp, CapChunkO
- Performed CPU profiling and optimization analysis:
  - Used Intel VTune
  - Analyzed threading overhead and runtime bottlenecks
- Collected and processed real-world sparse matrices:
  - Sourced from SuiteSparse Matrix Collection
- Conducted real-matrix benchmarking:
  - Evaluated performance on both CPU and GPU platforms
- Contributed to performance analysis and result interpretation (CPU and real datasets)

Joint Contributions

- Problem formulation and experimental design
- Integration of CPU and GPU pipelines
- Validation methodology (residual error checks)
- Comparative analysis with Intel oneMKL and NVIDIA cuSPARSE
- Writing, editing, and finalizing the project report

Tools and Assistance

- Generative AI tools (e.g., Claude) were used selectively for:
  - Improving clarity and structure of written sections
  - Assisting in interpretation of profiling outputs
- All core implementation, experimental design, and performance analysis were conducted independently by the authors

References

[1] I. S. Duff, A. M. Erisman, and J. K. Reid, "Direct methods for sparse matrices," Oxford University Press, 1986.

[2] J. R. Gilbert and R. E. Tarjan, "The analysis of a nested dissection algorithm," *Numerische Mathematik*, vol. 50, no. 4, pp. 377–404, 1992.

[3] M. Naumov, "Parallel solution of sparse triangular linear systems in the preconditioned iterative methods on the GPU," NVIDIA Technical Report NVR-2011-001, 2011.

[4] R. Vuduc, J. Demmel, and K. Yelick, "OSKI: A library of automatically tuned sparse matrix kernels," in *Proc. SciDAC*, 2005.

[5] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[6] Intel Corporation, "Intel oneAPI Math Kernel Library Developer Reference," 2024.

[7] NVIDIA Corporation, "cuSPARSE Library," 2024.

[8] Author, "Title of the reference," Year.

Appendix

This appendix provides the complete performance and residual data for all benchmarks evaluated in this work. The main-body results in Section 8 present condensed summaries; the tables below contain the full data for every matrix and kernel combination.

A. CPU Real-Matrix Performance and Speedup

Table IX reports the solve time (ms) and speedup relative to MKL for each custom CPU kernel across all 30 real matrices on the i9-14900HX. The $\times$MKL column denotes MKL time divided by kernel time.

B. CPU Real-Matrix Residuals

Table X reports relative residual norms for each CPU kernel on all 30 real matrices. All values lie in the range 10^{-16} to 10^{-11}, confirming numerical correctness.

C. GPU RTX 4070 Laptop — Real Matrices

Table XI reports LearnedAware and cuSPARSE solve times (ms) on the RTX 4070 for all 30 real matrices. The Speedup column is cuSPARSE/LearnedAware; values > 1.0 indicate LearnedAware is faster.

D. GPU RTX 4070 Laptop — Synthetic Families

E. GPU NVIDIA H200 — Real Matrices

Table XIII reports H200 results. The Ratio column is LearnedAware/cuSPARSE; values < 1.0 indicate LearnedAware is faster.

F. GPU NVIDIA H200 — Synthetic (Random Family)

Table XV reports all GPU kernel timings on H200 for the random synthetic family, showing the full progression from Coalesced through LearnedAware.

G. GPU NVIDIA H200 — Synthetic (Random Family)

Table XV reports all GPU kernel timings on H200 for the random synthetic family, showing the full progression from Coalesced through LearnedAware.

H. GPU Tesla P100 — Real Matrices

I. GPU Tesla P100 — Synthetic (Random Family)

This appendix describes how to reproduce all experiments.

J. Repository Structure

Listing 1. Repository layout

```
github/
|-- README.md
|-- src/
|   |-- cpu/
|   |   |-- sptrsv_benchmark.cpp
|   |   |-- sptrsv_kernel.cpp
|   |   '-- sptrsv_kernel.h
|   '-- gpu/
|       |-- sptrsv_benchmark.cu
|       |-- sptrsv_kernel.cu
|       |-- sptrsv_kernel.cuh
|       |-- sptrsv_kernel.h
|       '-- learned_selector_weights.h
|-- scripts/
|   |-- cpu/
```

TABLE IX
CPU REAL SWEEP: PERFORMANCE AND SPEEDUP OVER MKL (I9-14900HX)

Family	Matrix	N	NNZ	Basic	B×MKL	BranchRed	R×MKL	Unrolled	U×MKL	ThreshOmp	T×MKL	ChunkedOmp	C×MKL	CapChunkO	CC×MKL	MKL
random_irreg.	ASIC_100k	99340	526789	1.083	1.45	1.029	1.53	1.044	1.51	1.588	0.99	1.528	1.03	1.356	1.16	1.575
banded	LFAT5000	19994	49980	0.050	1.22	0.071	0.86	0.092	0.66	0.091	0.67	0.077	0.79	0.085	0.72	0.061
chain_like	poli	4008	4348	0.007	1.14	0.008	1.03	0.009	0.91	0.013	0.63	0.013	0.64	0.013	0.63	0.008
wide_level	web-Stanford	281903	1413271	3.139	1.16	3.544	1.03	3.990	0.91	4.739	0.77	4.004	0.91	3.949	0.92	3.638
block_struct.	circuit204	1020	3795	0.0047	1.36	0.0049	1.31	0.0050	1.26	0.0072	0.87	0.0068	0.93	0.0073	0.87	0.0063
power_law	soc-sign-epin.	131828	593597	1.316	0.91	1.199	0.99	1.273	0.94	1.802	0.66	1.746	0.68	1.545	0.77	1.193
extra	msc04515	4515	51111	0.040	0.94	0.034	1.10	0.035	1.10	0.061	0.62	0.070	0.54	0.077	0.49	0.038
extra	pwtk	217918	5926171	3.797	1.16	4.553	0.97	4.106	1.07	4.773	0.92	4.668	0.94	4.664	0.95	4.411
extra	apache1	80800	311492	0.509	0.99	0.499	1.01	0.560	0.90	0.523	0.96	0.517	0.97	0.571	0.88	0.502
extra	ASIC_680k	682862	2277355	2.598	1.66	2.751	1.57	2.791	1.54	5.899	0.73	5.174	0.83	6.566	0.66	4.312
very_large	web-BerkStan	685230	4425244	5.606	1.38	5.866	1.32	5.972	1.30	10.192	0.76	12.193	0.63	9.641	0.80	7.742
very_large	roadNet-TX	1393383	3315043	9.195	1.35	9.413	1.32	9.131	1.36	20.341	0.61	26.460	0.47	27.052	0.46	12.429
very_large	wiki-Talk	2394385	3072221	9.771	1.81	9.959	1.77	16.161	1.09	19.345	0.91	21.287	0.83	24.161	0.73	17.670

TABLE X
CPU REAL SWEEP: RESIDUAL NORMS ($\|Lx - b\|_\infty / \|b\|_\infty$)

Family	Matrix	N	NNZ	Basic	BranchRed	Unrolled	ThreshOmp	ChunkedOmp	CapChunkO	MKL
random_irreg.	ASIC_100k	99340	526789	1.92e-14	1.92e-14	1.39e-14	1.39e-14	1.39e-14	1.39e-14	1.52e-14
banded	LFAT5000	19994	49980	2.37e-11	2.37e-11	2.37e-11	2.37e-11	2.37e-11	2.37e-11	3.27e-11
chain_like	poli	4008	4348	7.44e-16	7.44e-16	7.23e-16	7.23e-16	7.23e-16	7.23e-16	3.66e-15
wide_level	web-Stanford	281903	1413271	2.76e-14	2.76e-14	1.94e-14	1.94e-14	1.94e-14	1.94e-14	2.05e-14
block_struct.	circuit204	1020	3795	1.22e-15	1.22e-15	7.67e-16	7.67e-16	7.67e-16	7.67e-16	1.61e-15
power_law	soc-sign-epin.	131828	593597	1.69e-14	1.69e-14	9.65e-15	9.65e-15	9.65e-15	9.65e-15	1.25e-14
extra	msc04515	4515	51111	8.34e-15	8.34e-15	6.95e-15	6.95e-15	6.95e-15	6.95e-15	5.66e-15
extra	pwtk	217918	5926171	6.66e-14	6.66e-14	4.04e-14	4.04e-14	4.04e-14	4.04e-14	2.50e-14
extra	helm2d03	392257	1567096	3.63e-14	3.63e-14	2.73e-14	2.73e-14	2.73e-14	2.73e-14	2.88e-14
extra	apache1	80800	311492	1.66e-14	1.66e-14	1.09e-14	1.09e-14	1.09e-14	1.09e-14	1.27e-14
extra	bcsstk13	2003	42943	1.74e-14	1.74e-14	1.83e-14	1.83e-14	1.83e-14	1.83e-14	1.12e-14
extra	bcsstk14	1806	32630	1.62e-14	1.62e-14	2.62e-14	2.62e-14	2.62e-14	2.62e-14	1.45e-14
extra	bcsstk17	10974	219812	1.64e-14	1.64e-14	1.25e-14	1.25e-14	1.25e-14	1.25e-14	7.61e-15
extra	bcsstk18	11948	80519	4.95e-14	4.95e-14	5.61e-14	5.61e-14	5.61e-14	5.61e-14	4.78e-14
extra	bcsstk27	1224	28675	7.86e-15	7.86e-15	6.89e-15	6.89e-15	6.89e-15	6.89e-15	5.29e-15
extra	bcsstm22	138	138	2.85e-16	2.85e-16	2.85e-16	2.85e-16	2.85e-16	2.85e-16	4.39e-16
extra	scircuit	170998	565030	2.11e-14	2.11e-14	1.75e-14	1.75e-14	1.75e-14	1.75e-14	1.76e-14
extra	fv1	9604	47434	5.97e-15	5.97e-15	2.88e-15	2.88e-15	2.88e-15	2.88e-15	2.92e-15
extra	fv2	9801	48413	6.13e-15	6.13e-15	3.05e-15	3.05e-15	3.05e-15	3.05e-15	3.68e-15
extra	fv3	9801	48413	6.07e-15	6.07e-15	3.10e-15	3.10e-15	3.10e-15	3.10e-15	3.95e-15
extra	EPA	4772	7138	2.33e-15	2.33e-15	2.10e-15	2.10e-15	2.10e-15	2.10e-15	3.64e-15
extra	ca-GrQc	5242	19726	3.74e-15	3.74e-15	3.04e-15	3.04e-15	3.04e-15	3.04e-15	3.57e-15
extra	email-Enron	36692	220523	1.19e-14	1.19e-14	7.85e-15	7.85e-15	7.85e-15	7.85e-15	1.02e-14
extra	soc-Epinions1	75888	363549	1.57e-14	1.57e-14	1.10e-14	1.10e-14	1.10e-14	1.10e-14	7.86e-15
extra	roadNet-CA	1971281	4737888	4.73e-14	4.73e-14	4.23e-14	4.23e-14	4.23e-14	4.23e-14	9.74e-14
extra	wiki-Vote	8297	40953	4.33e-15	4.33e-15	2.72e-15	2.72e-15	2.72e-15	2.72e-15	2.69e-15
extra	ASIC_680k	682862	2277355	3.02e-14	3.02e-14	2.93e-14	2.93e-14	2.93e-14	2.93e-14	3.39e-14
very_large	web-BerkStan	685230	4425244	5.46e-14	5.46e-14	3.92e-14	3.92e-14	3.92e-14	3.92e-14	3.21e-14
very_large	roadNet-TX	1393383	3315043	5.15e-14	5.15e-14	4.83e-14	4.83e-14	4.83e-14	4.83e-14	6.01e-14
very_large	wiki-Talk	2394385	3072221	4.66e-14	4.66e-14	4.39e-14	4.39e-14	4.39e-14	4.39e-14	8.29e-14

```
|   |   |-- run_benchmark.sh
|   |   |-- run_cpu_solve_only_tables.sh
|   |   |-- run_vtune_per_kernel_profiles.sh
|   |   '-- run_vtune_all_real_profiles.sh
|   |-- gpu/
|   |   |-- run_benchmark.sh
|   |   '-- run_local_full_profile.sh
|   '-- explorer/
|       |-- cpu/
|       |   |-- CPU_Run_Timings.script
|       |   '-- CPU_Run_Profile.script
|       '-- gpu/
|           |-- p100/
|           |-- v100/
|           '-- h200/
|-- data/matrices/
|-- results/
'-- docs/
```

K. Requirements

- GNU or compatible C++ compiler with OpenMP support
- Intel oneMKL (discoverable via `MKLROOT`)
- CUDA toolkit for GPU builds
- Optional: Intel VTune (CPU profiling), Nsight Systems (GPU profiling)

L. Local CPU Runs

Listing 2. CPU benchmarks

```
cd github
```

TABLE XI
GPU RTX 4070 MOBILE: REAL-MATRIX SWEEP

Family	Matrix	N	NNZ	LearnedAware (ms)	cuSPARSE (ms)	Speedup
random_irreg.	ASIC_100k	99340	526789	11.31	37.98	3.36
banded	LFAT5000	19994	49980	11.83	22.24	1.88
chain_like	poli	4008	4348	1.60	1.73	1.08
wide_level	web-Stanford	281903	1413271	32.83	20.02	0.61
block_struct.	circuit204	1020	3795	1.49	1.38	0.92
power_law	soc-sign-epin.	131828	593597	34.97	14.27	0.41
extra	msc04515	4515	51111	5.91	3.74	0.63
extra	pwtk	217918	5926171	541.00	995.12	1.84
extra	helm2d03	392257	1567096	22.95	20.81	0.91
extra	apache1	80800	311492	63.67	5.96	0.09
extra	bcsstk13	2003	42943	5.03	5.86	1.16
extra	bcsstk14	1806	32630	5.99	7.19	1.20
extra	bcsstk17	10974	219812	46.15	23.85	0.52
extra	bcsstk18	11948	80519	94.41	4.34	0.05
extra	bcsstk27	1224	28675	3.98	8.22	2.06
extra	bcsstm22	138	138	2.98	3.68	1.23
extra	scircuit	170998	565030	180.70	13.79	0.08
extra	fv1	9604	47434	8.18	2.72	0.33
extra	fv2	9801	48413	24.84	5.00	0.20
extra	fv3	9801	48413	12.99	4.16	0.32
extra	EPA	4772	7138	6.95	2.77	0.40
extra	ca-GrQc	5242	19726	12.90	1.49	0.12
extra	email-Enron	36692	220523	29.14	10.41	0.36
extra	soc-Epinions1	75888	363549	24.45	11.35	0.46
extra	roadNet-CA	1971281	4737888	145.93	38.10	0.26
extra	wiki-Vote	8297	40953	24.82	3.65	0.15
extra	ASIC_680k	682862	2277355	132.77	259.89	1.96
very_large	web-BerkStan	685230	4425244	264.13	181.27	0.69
very_large	roadNet-TX	1393383	3315043	112.13	38.43	0.34
very_large	wiki-Talk	2394385	3072221	113.87	80.92	0.71

```
./scripts/cpu/run_benchmark.sh real
./scripts/cpu/run_benchmark.sh families
```

M. Local GPU Runs

Listing 3. GPU benchmarks (RTX 4070 example)

```
cd github
CUDA_ARCH=89 ./scripts/gpu/run_benchmark.sh real
CUDA_ARCH=89 ./scripts/gpu/run_benchmark.sh families
```

Replace `89` with the compute capability of your GPU (`60` for P100, `90` for H200).

N. Explorer CPU Runs

Listing 4. Explorer CPU batch jobs

```
module load intel mkl
cd github
sbatch scripts/explorer/cpu/CPU_Run_Timings.script
sbatch scripts/explorer/cpu/CPU_Run_Profile.script
```

O. Explorer GPU Runs

Listing 5. Explorer GPU batch jobs

```
module load cuda/12.8
cd github
# Tesla P100
sbatch scripts/explorer/gpu/p100/GPU_Run_Timings.script
# NVIDIA H200
sbatch scripts/explorer/gpu/h200/GPU_Run_Timings.script
```

P. Profiling

Listing 6. Profiling commands

```
# CPU: VTune hotspot analysis
./scripts/cpu/run_vtune_per_kernel_profiles.sh
```

TABLE XII
GPU RTX 4070 MOBILE: SYNTHETIC SWEEP

Family	N	NNZ	LearnedAware (ms)	cuSPARSE (ms)	Speedup
random	1024	6228	2.87	34.36	11.97
random	4096	87943	3.44	2.98	0.87
random	16384	1358374	17.46	11.50	0.66
random	32768	5401584	49.45	36.25	0.73
banded	1024	64480	3.25	7.97	2.46
banded	4096	264160	8.72	31.96	3.66
banded	16384	1062880	33.44	106.56	3.19
banded	32768	2127840	66.31	218.00	3.29
chain	1024	2047	1.51	4.25	2.82
chain	4096	8191	2.71	14.37	5.29
chain	16384	32767	8.16	50.53	6.19
chain	32768	65535	14.55	99.85	6.86
wide-level	1024	33264	1.40	2.96	2.12
wide-level	4096	134640	2.09	9.30	4.44
wide-level	16384	540144	4.97	43.65	8.79
wide-level	32768	1080816	7.35	63.07	8.58
block	1024	23048	9.70	3.56	0.37
block	4096	108193	35.41	11.91	0.34
block	16384	686696	44.41	58.54	1.32
block	32768	2044383	78.38	149.47	1.91
power-law	1024	1507	1.09	1.01	0.93
power-law	4096	8266	1.45	0.97	0.67
power-law	16384	49929	2.62	1.61	0.61
power-law	32768	127933	4.16	2.23	0.54

```
# Explorer CPU profiling
sbatch scripts/explorer/cpu/CPU_Run_Profile.script

# GPU: Nsight Systems
./scripts/gpu/run_local_full_profile.sh
```

TABLE XIII
GPU NVIDIA H200: REAL-MATRIX SWEEP

Family	Matrix	N	NNZ	AutoFam	AutoKern	FamilyAware	LearnedAware	cuSPARSE	Ratio
random_irreg.	ASIC_100k	99340	526789	random	RandomAware	132.30	7.93	39.55	0.20
banded	LFAT5000	19994	49980	banded	RegularAware	12.24	8.90	26.03	0.34
chain_like	poli	4008	4348	power-law	PowerLawAware	0.40	0.33	1.35	0.25
wide_level	web-Stanford	281903	1413271	random	RandomAware	16.13	22.37	20.12	1.11
block_struct.	circuit204	1020	3795	power-law	PowerLawAware	0.48	0.40	1.46	0.27
power_law	soc-sign-epin.	131828	593597	random	RandomAware	12.35	17.07	18.40	0.93
extra	msc04515	4515	51111	random	RandomAware	1.98	3.62	3.29	1.10
extra	pwtk	217918	5926171	block	RowBucketed	221.38	354.88	954.68	0.37
extra	helm2d03	392257	1567096	power-law	PowerLawAware	10.09	13.02	4.81	2.71
extra	apache1	80800	311492	regular	RegularAware	52.66	42.69	3.61	11.84
extra	bcsstk13	2003	42943	block	RowBucketed	2.90	3.09	5.85	0.53
extra	bcsstk14	1806	32630	random	RandomAware	2.50	1.79	4.25	0.42
extra	bcsstk17	10974	219812	random	RandomAware	8.30	15.68	12.75	1.23
extra	bcsstk18	11948	80519	random	RandomAware	3.07	14.33	3.85	3.72
extra	bcsstk27	1224	28675	random	RandomAware	2.74	1.21	4.62	0.26
extra	bcsstm22	138	138	chain	RowBucketed	0.25	0.22	1.21	0.18
extra	scircuit	170998	565030	power-law	PowerLawAware	19.81	21.86	10.96	1.99
extra	fv1	9604	47434	regular	RegularAware	6.62	7.06	2.86	2.47
extra	fv2	9801	48413	regular	RegularAware	6.74	7.20	2.90	2.48
extra	fv3	9801	48413	regular	RegularAware	6.73	7.18	2.88	2.49
extra	EPA	4772	7138	power-law	PowerLawAware	0.47	0.46	1.32	0.35
extra	ca-GrQc	5242	19726	power-law	PowerLawAware	1.09	1.16	1.67	0.69
extra	email-Enron	36692	220523	random	RandomAware	5.77	9.16	12.78	0.72
extra	soc-Epinions1	75888	363549	random	RandomAware	7.68	10.91	13.06	0.84
extra	roadNet-CA	1971281	4737888	random	RandomAware	75.05	105.38	15.58	6.77
extra	wiki-Vote	8297	40953	random	RandomAware	2.31	3.07	3.48	0.88
extra	ASIC_680k	682862	2277355	power-law	PowerLawAware	17.18	22.56	92.30	0.24
very_large	web-BerkStan	685230	4425244	random	RandomAware	55.76	92.73	121.29	0.76
very_large	roadNet-TX	1393383	3315043	random	RandomAware	53.64	74.45	10.83	6.87
very_large	wiki-Talk	2394385	3072221	power-law	PowerLawAware	46.96	60.12	44.71	1.34

TABLE XIV
GPU H200: SYNTHETIC RANDOM FAMILY — ALL KERNELS (MS)

Fam.	N	NNZ	AutoF	AutoK	Coal.	RowBkt	LvlAw	RndAw	PwrAw	BlkAw	BndAw	FamAw	LrnAw	cuSP
rand	1024	6228	rand	RndAw	132.67	1.31	0.74	0.53	0.53	2.36	1.31	0.45	0.59	18.87
rand	4096	87943	rand	RndAw	5.77	5.55	2.16	1.01	1.71	22.20	5.64	1.26	1.37	3.98
rand	16384	1358374	rand	RndAw	25.14	24.77	7.70	4.75	6.75	452.54	25.83	10.51	10.51	10.76
rand	32768	5401584	rand	RndAw	64.28	62.93	21.21	16.66	20.06	2708.36	66.37	26.81	43.98	31.51

TABLE XV
GPU H200: SYNTHETIC RANDOM FAMILY — ALL KERNELS (MS)

Fam.	N	NNZ	AutoF	AutoK	Coal.	RowBkt	LvlAw	RndAw	PwrAw	BlkAw	BndAw	FamAw	LrnAw	cuSP
rand	1024	6228	rand	RndAw	132.67	1.31	0.74	0.53	0.53	2.36	1.31	0.45	0.59	18.87
rand	4096	87943	rand	RndAw	5.77	5.55	2.16	1.01	1.71	22.20	5.64	1.26	1.37	3.98
rand	16384	1358374	rand	RndAw	25.14	24.77	7.70	4.75	6.75	452.54	25.83	10.51	10.51	10.76
rand	32768	5401584	rand	RndAw	64.28	62.93	21.21	16.66	20.06	2708.36	66.37	26.81	43.98	31.51

TABLE XVI
GPU TESLA P100: REAL-MATRIX SWEEP

Family	Matrix	N	NNZ	AutoFam	AutoKern	FamilyAware	LearnedAware	cuSPARSE	Ratio
random_irreg.	ASIC_100k	99340	526789	random	RandomAware	167.74	10.09	56.61	0.18
banded	LFAT5000	19994	49980	banded	RegularAware	21.42	19.99	42.43	0.47
chain_like	poli	4008	4348	power-law	PowerLawAware	0.52	0.50	0.62	0.81
wide_level	web-Stanford	281903	1413271	random	RandomAware	22.17	29.38	26.23	1.12
block_struct.	circuit204	1020	3795	power-law	PowerLawAware	0.55	0.55	0.72	0.76
power_law	soc-sign-epin.	131828	593597	random	RandomAware	16.73	21.24	24.02	0.88
extra	msc04515	4515	51111	random	RandomAware	2.22	6.08	3.65	1.67
extra	pwtk	217918	5926171	block	RowBucketed	384.30	756.31	1560.86	0.48
extra	helm2d03	392257	1567096	power-law	PowerLawAware	16.46	20.79	9.61	2.16
extra	apache1	80800	311492	regular	RegularAware	108.96	94.82	5.50	17.25
extra	bcsstk13	2003	42943	block	RowBucketed	4.55	6.56	7.27	0.90
extra	bcsstk14	1806	32630	random	RandomAware	2.77	2.89	4.93	0.59
extra	bcsstk17	10974	219812	random	RandomAware	9.48	34.26	17.36	1.97
extra	bcsstk18	11948	80519	random	RandomAware	3.86	20.89	4.51	4.64
extra	bcsstk27	1224	28675	random	RandomAware	3.01	2.00	5.57	0.36
extra	bcsstm22	138	138	chain	RowBucketed	0.36	0.34	0.52	0.65
extra	scircuit	170998	565030	power-law	PowerLawAware	25.06	27.91	19.35	1.44
extra	fv1	9604	47434	regular	RegularAware	11.31	11.76	3.13	3.76
extra	fv2	9801	48413	regular	RegularAware	11.56	11.98	3.15	3.81
extra	fv3	9801	48413	regular	RegularAware	11.56	11.98	3.16	3.79
extra	EPA	4772	7138	power-law	PowerLawAware	0.58	0.63	0.64	0.99
extra	ca-GrQc	5242	19726	power-law	PowerLawAware	1.29	1.43	1.17	1.23
extra	email-Enron	36692	220523	random	RandomAware	7.66	11.02	13.82	0.80
extra	soc-Epinions1	75888	363549	random	RandomAware	10.49	13.48	16.62	0.81
extra	roadNet-CA	1971281	4737888	random	RandomAware	100.45	138.50	38.91	3.56
extra	wiki-Vote	8297	40953	random	RandomAware	3.00	3.63	3.33	1.09
extra	ASIC_680k	682862	2277355	power-law	PowerLawAware	30.74	37.67	125.13	0.30
very_large	web-BerkStan	685230	4425244	random	RandomAware	87.81	123.34	166.17	0.74
very_large	roadNet-TX	1393383	3315043	random	RandomAware	71.87	98.70	28.84	3.42
very_large	wiki-Talk	2394385	3072221	power-law	PowerLawAware	76.33	93.30	68.67	1.36

TABLE XVII
GPU TESLA P100: SYNTHETIC RANDOM FAMILY — ALL KERNELS (MS)

Fam.	N	NNZ	AutoF	AutoK	Coal.	RowBkt	LvlAw	RndAw	PwrAw	BlkAw	BndAw	FamAw	LrnAw	cuSP
rand	1024	6228	rand	RndAw	171.36	2.07	0.97	4.00	0.68	4.10	2.04	0.64	0.77	30.84
rand	4096	87943	rand	RndAw	13.70	9.41	2.65	1.31	2.13	42.17	9.62	1.58	1.87	2.34
rand	16384	1358374	rand	RndAw	63.22	58.97	10.58	8.20	9.78	632.09	55.20	11.28	14.41	14.67
rand	32768	5401584	rand	RndAw	141.56	130.19	28.72	23.67	27.06	3679.24	130.07	35.57	58.89	46.15